

Prereq to Distributed Training

Outline

- PyTorch System Architecture
- CUDA Fundamentals
- Mixed Precision
- Large Scale Data

Notes: the content is still being developed.

PyTorch System Architecture: From API to Hardware

PyTorch System Architecture

High-level API

PyTorch API

Intermediate/runtime Layer

ATen (C++ Tensor Library)

AutoGrad Engine

Dispatch System

Distributed (FSDP, DDP)

TorchScript

Device Backend (Per Hardware)

CPU backend
(MLK/oneDNN, ...)

CUDA Backend
(cuBLAS, cuDNN, .)

ROCm Backend
(rocBLAS, MIOpen,)

Triton

Compiler/Kernel Level

LLVM
(CPU JIT, XLA, MLIR Backend)

NVCC/NVRTC
(CUDA Kernel compilation)

HIP/ROC compiler
(AMD)

Driver & Hardware

CUDA driver + runtime API

ROCm Runtime + HSA (AMD)

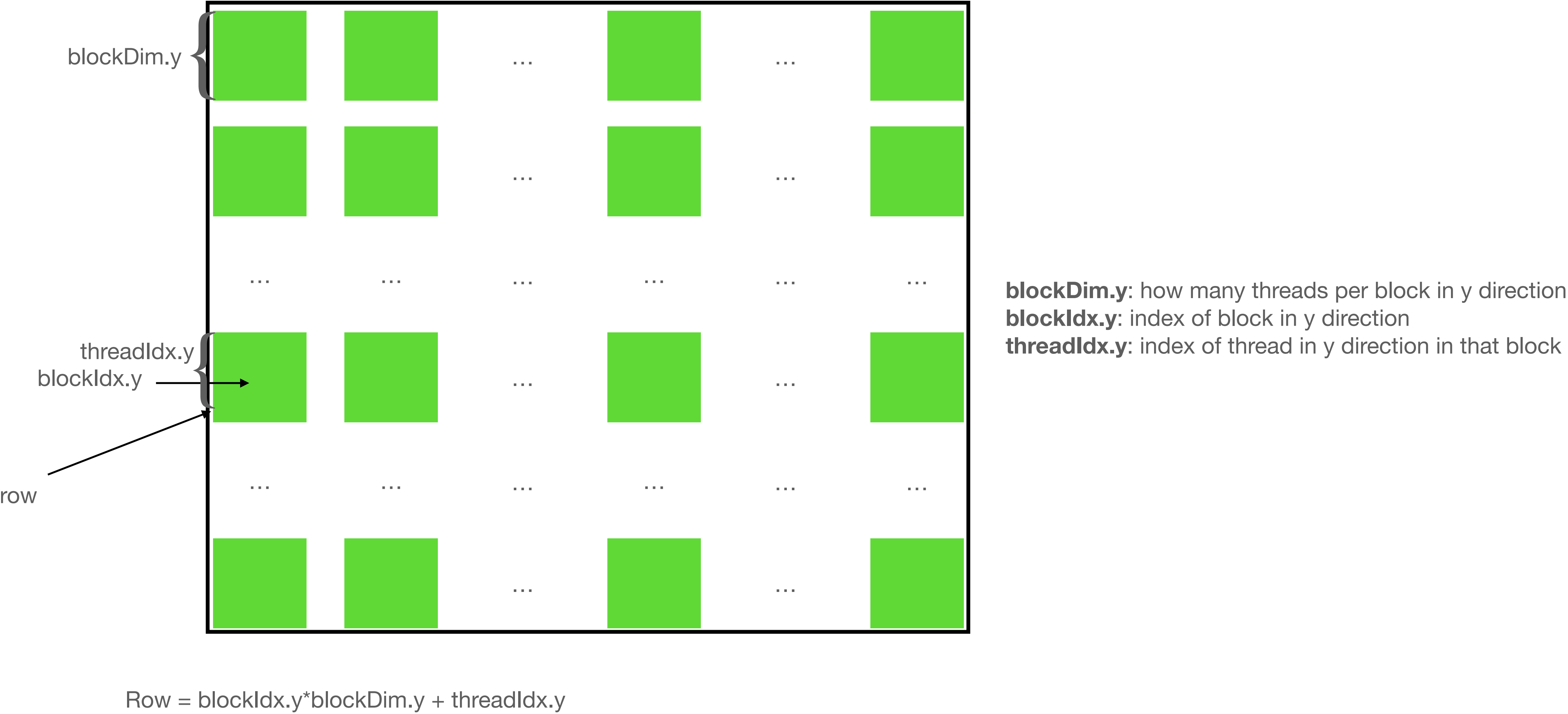
Direct CPU instruction (AVX,)

PyTorch System Architecture

Layer	Role	Key Tech
High level Framework	Interface for defining models, training loops, etc.	Python, torch.nn, torch.optim, ...
Intermediate/runtime Layer	Core tensor operation, gradient computation, Operation dispatch.	ATen, C10 Dispatcher, autograd, pybind11, ...
Device Backend	Provide implementation of ops for each device	C++/CUDA/HIP kernel, ...
Compiler/kernel Level	Compiles and optimizes code to run efficiently on specific hardware	NVCC, LLVM, XLA, cuDNN, ...
Driver/Hardware	Executes compiled kernels on actual hardware	CUDA driver, ROCm, CPU driver, ...

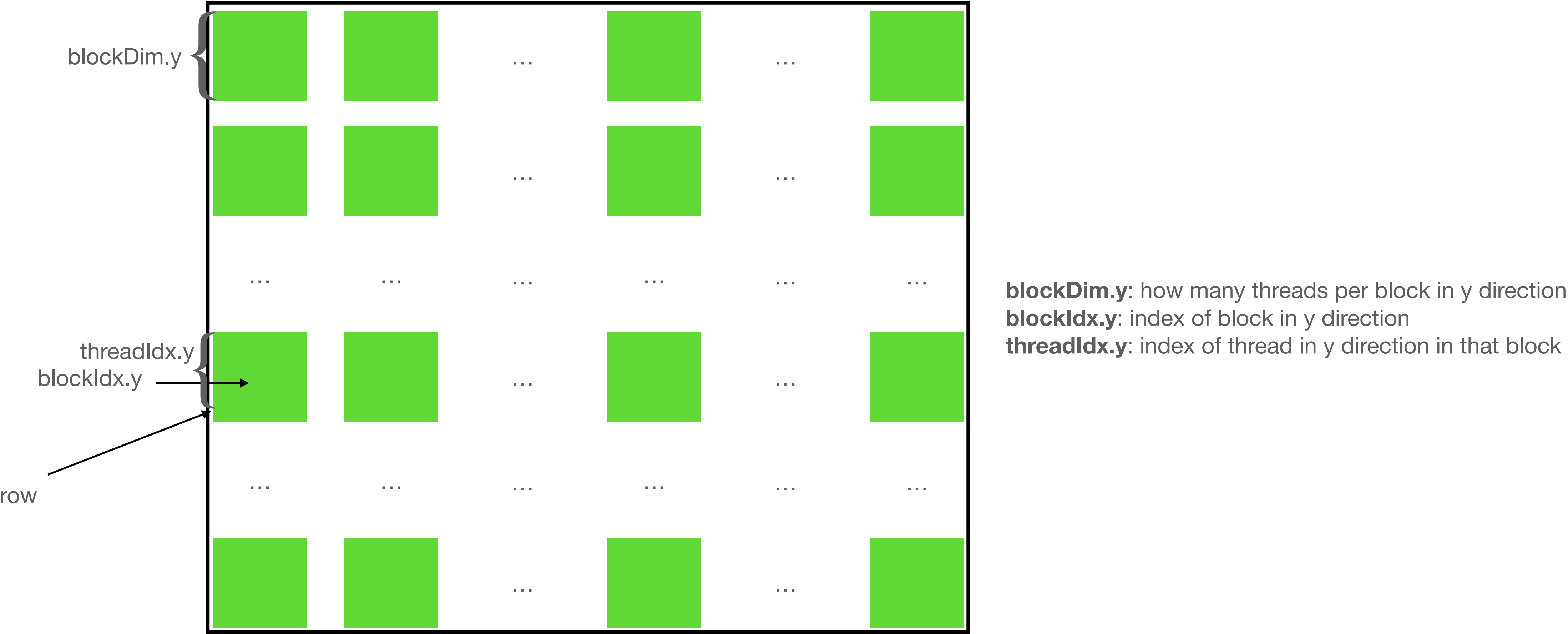
CUDA Fundamentals

How threads and blocks are organized



CUDA Fundamentals

How threads and blocks are organized

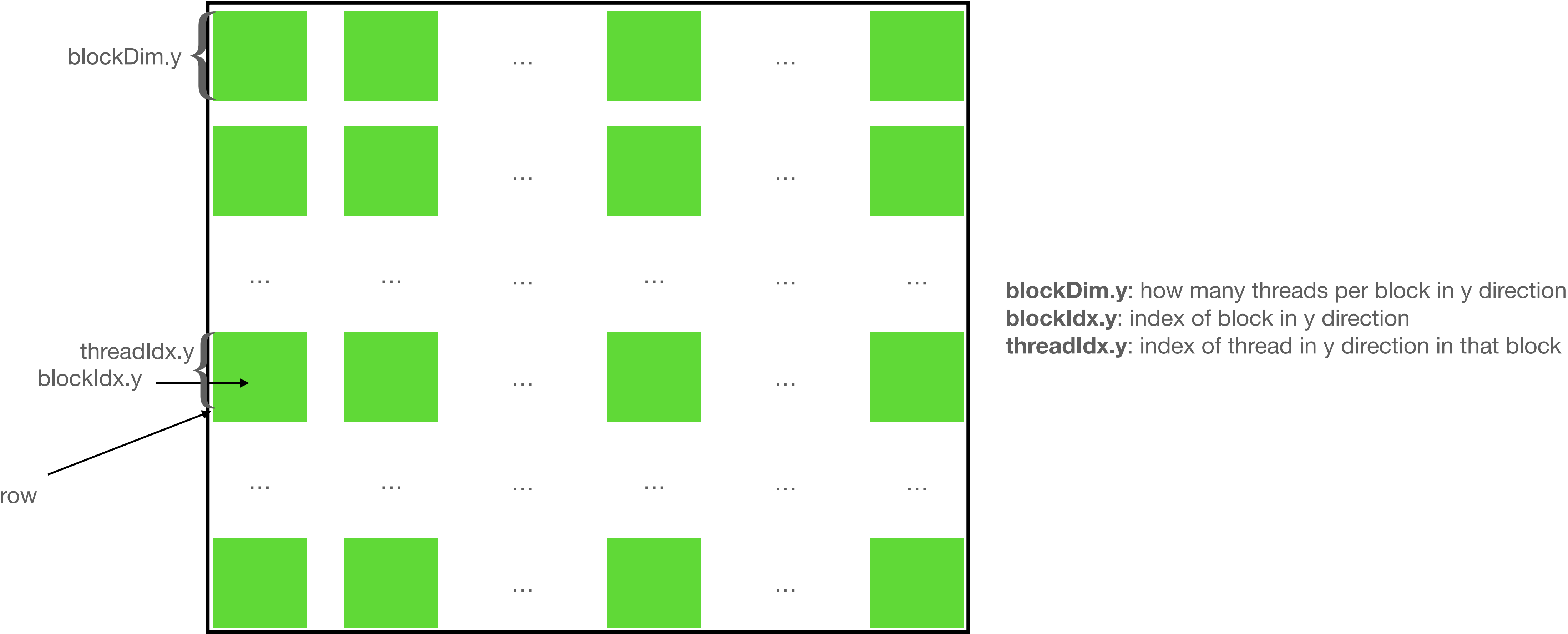


```
dim3 threadsPerblock(16, 16)
```

tells 16*16 threads per block

CUDA Fundamentals

How threads and blocks are organized



```
dim3 numBlocks(128, 128)
```

 tells 128*128 blocks totally.

CUDA Fundamentals

CUDA Fundamentals

Why?

```
dim3 threadsPerblock(16, 16)
```

Rule of thumb:

1. That **blockSize** should ideally be a multiple of 32, because **GPU hardware executes in warps of 32 threads**.
2. Start with 16*16 (256 threads).
3. Profile with nvprof and nsys and try 32*8, 8*32, or may be 32*16.

`blockDim.x * blockDim.y * blockDim.z <= 1024` means the max number of threads per block is 1024.

CUDA Fundamentals

Matrix Multiplication Naive Version

```
// CUDA kernel for matrix multiplication
__global__ void matrixMulKernel(float* A, float* B, float* C, int width) {
    int row = blockIdx.y * blockDim.y + threadIdx.y; // Row index of C
    int col = blockIdx.x * blockDim.x + threadIdx.x; // Column index of C

    if (row < width && col < width) {
        float value = 0;
        for (int k = 0; k < width; ++k) {
            value += A[row * width + k] * B[k * width + col];
        }
        C[row * width + col] = value;
    }
}
```

Why is it naive version:

1. Each thread reads global memory for every $A[\text{row}, k]$ and $B[k, \text{col}]$.
2. Global memory accesses are slow (400~600 cycles).
3. Redundant loads: multiple threads read the same element of A and B repeatedly.

CUDA Fundamentals

Matrix Multiplication Tiled Version

```
// CUDA kernel for tiled matrix multiplication
__global__ void matrixMulKernelTiled(float* A, float* B, float* C, int width) {
    // Shared memory for sub-matrices (tiles) of A and B
    __shared__ float shared_A[TILE_SIZE][TILE_SIZE];
    __shared__ float shared_B[TILE_SIZE][TILE_SIZE];

    // Thread row and column in the output matrix
    int row = blockIdx.y * TILE_SIZE + threadIdx.y;
    int col = blockIdx.x * TILE_SIZE + threadIdx.x;
    float value = 0;

    // Loop over tiles of A and B
    for (int t = 0; t < (width / TILE_SIZE); ++t) {
        // Load data into shared memory
        shared_A[threadIdx.y][threadIdx.x] = A[row * width + (t * TILE_SIZE + threadIdx.x)];
        shared_B[threadIdx.y][threadIdx.x] = B[(t * TILE_SIZE + threadIdx.y) * width + col];

        __syncthreads(); // Synchronize threads to ensure that the tiles are loaded before proceeding

        // Perform partial multiplication for this tile
        for (int k = 0; k < TILE_SIZE; ++k) {
            value += shared_A[threadIdx.y][k] * shared_B[k][threadIdx.x];
        }

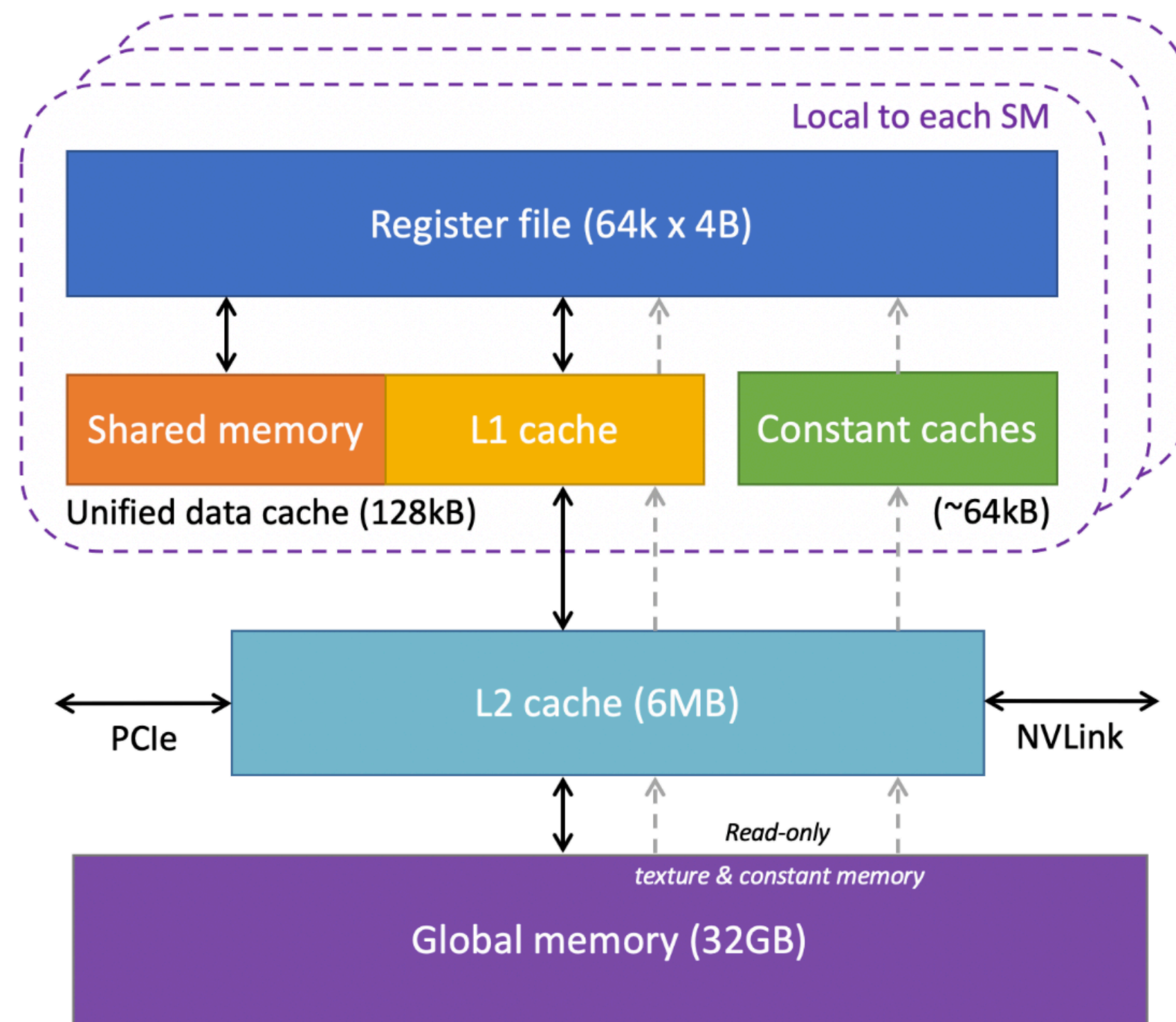
        __syncthreads(); // Synchronize threads before loading the next tile
    }

    // Write result to the output matrix C
    if (row < width && col < width) {
        C[row * width + col] = value;
    }
}
```

1. Loop over tiles of k dimension.
2. Load tiles of A and B into fast shared memory.
3. Sync threads.
4. Compute partial sum for this tile.
5. Sync again.
6. Repeat until all tiles are processed.

CUDA Fundamentals

GPU Memory Hierarchy



GPU memory levels and sizes for the NVIDIA Tesla V100.
(Based on diagrams by [NVIDIA](#) and [Citadel](#) at GTC 2018)

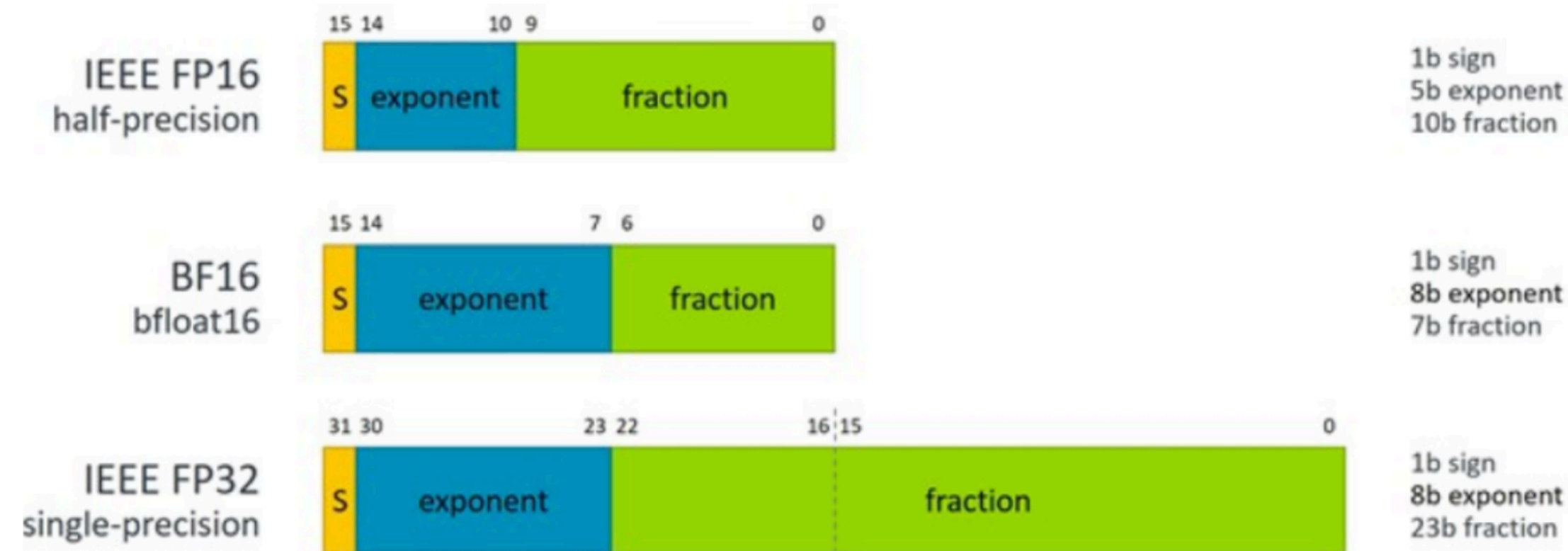
CUDA Fundamentals

GPU Memory Hierarchy

	Scope	Location	Speed	Size	Usage example
Register	Per-thread, private	Inside SM	1 cycle	Tens of KB per SM	Storing thread local scalar, ...
Shared Memory	Per-thread block	Inside SM	10s cycles	~48-100KB per SM	Matrix tiles for gemm
L1 Cache	per-SM	On chip	20-30 cycles	~128KB per SM	Automatic caching
L2 Cache	All SM	On chip	~200 cycles	~6-96 MB	Global memory staging
Global Memory	All threads all SM	On chip	~400-800 cycles	GBs	Managing working memory

Mixed Precision

Mixed Precision



- **Floating point representation:** $\text{fraction (mantissa)} \times \text{base}^{\text{exponent}}$
 - Mantissa: the significant digits of number, determining precision
 - Exponent: the number magnitude.
- Example, a large number $1e6$ and a small number $1e-7$
 - FP16, max = 65000, so $1e6$ just overflow, becomes infinity, $1e-7$ too small, becomes 0
 - BF16, max = $3.4e38$, min = $1e-38$, both $1e6$ and $1e-7$ are in the range, but only 7 mantissa bit, probably rounding 1.23456 to 1.23

Mixed Precision

- **Speed**
 - Modern GPUs/TPUs have Tensor cores optimized for FP16/BF16
 - 2-8x faster matmul/convolution
- **Memory Saving**
 - Half the memory of FP32
 - Enable larger batch sizes/models
- **Accuracy**
 - Same or better accuracy
 - Need proper handling
 - Better regularization effect

Mixed Precision

- **Training (Practical Guidance)**
 - Forward pass
 - Activation, matmul, conv in FP16/BF16 (Fast, memory efficient)
 - Sensitive ops (softmax, normalization, reduction) in FP32
 - Backward pass
 - Gradient in FP16/BF16.
 - Accumulated in FP32 to avoid precision loss.
 - Weights
 - Master weights in FP32 (for stable updates)
 - Cast to FP16/BF16 for forward/backward.
 - Loss Scaling
 - Dynamic or static scaling to avoid underflow.
- **Inference (Practical Guidance)**
 - FP32, input, weight scale, scalar.
 - BF16, Embedding weight, embedding norm weight, vector.
 - Float8_e4m3fn, matrix weights, matrix, matrix multiplication.

Large Scale Data

How to handle large file?

- Problem: One data file containing several billions training examples with labels, does not fit into RAM, how to deal with this file?
- Solution: sequentially scan the file line by line and process each line individually.

```
with open("data.jsonl", "r") as file:  
    for line in file:  
        x, y = json.load(line)
```

- The following will trigger the memory overflow since it will load all example into RAM.

```
with open("data.jsonl", "r") as file:  
    data = file.read()  
  
with open("data.jsonl", "r") as file:  
    data = [json.load(line) for line in f]
```

How to sample from large file?

- Problem: Given the same previous large data file containing several billions training examples with labels, how can we uniformly sample K examples from the file?
- Solution:
 - Naive approach: First determine the total number of examples (as described previously); then randomly select K indices; perform a second scan over the file to extract the examples corresponding to those indices.
 - Reservoir sampling

```
def reservoirSampling(stream, K):  
    reservior = []  
    for i, item in enumerate(stream):  
        if i < K:  
            reservior.append(item)  
        else:  
            j = random.randint(0, i)  
            if j < k:  
                reservior[j] = item  
    return reservior
```

How to generate batch data?

- Problem: Given the data, how to generate batch during training or eval stage?

```
def generate_batch(data, batch_size, seed=None, start_index=0):  
    total = len(data)  
    indices = np.arange(total)  
  
    # shuffle once using the provided seed  
    if seed is not None:  
        rng = np.random.default_rng(seed)  
        rng.shuffle(indices)  
    else:  
        np.random.shuffle(indices)  
  
    # yield batches starting from start_index  
    for start in range(start_index, total, batch_size):  
        end = min(start + batch_size, total)  
        batch_indices = indices[start:end]  
        yield [data[i], for i in batch_indices]
```

- Note: 1. Given the seed, produce the same identical batch sequences; 2. Resume from given batch start index.

How to generate batch data?

- Problem: similar to the previous, but the data does not fit into Memory, how to generate batch during training or eval stage?
 - If we have multiple small data files

```
def generate_batch_from_data_files(data_files, batch_size, seed=None, start_index=0):  
  
    total = len(data_files)  
    indices = np.arange(total)  
  
    # shuffle once using the provided seed  
    if seed is not None:  
        rng = np.random.default_rng(seed)  
        rng.shuffle(indices)  
    else:  
        np.random.shuffle(indices)  
  
    # yield batches starting from start_index  
    for start in range(start_index, total, batch_size):  
        end = min(start + batch_size, total)  
        batch_indices = indices[start:end]  
        batch = []  
        for i in batch_indices:  
            with open(data_files[i], "r") as f:  
                batch.append(json.load(f))  
        yield batch
```

How to generate batch data?

- Problem: similar to the previous, but the data does not fit into Memory, how to generate batch during training or eval stage?
 - If we have only one large file

```
def generate_batches_jsonl(file_path, batch_size):  
    batch = []  
    with open(file_path, "r") as file:  
        for line in f:  
            item = json.load(line)  
            batch.append(item)  
            if len(batch) == batch_size:  
                yield batch  
                batch = []  
    if batch:  
        yield batch
```

How to go faster on generating batch data?

- **General Recommendation:**
 - **Multiprocessing:** CPU-bound, batches requires heavy computation (e.g, parsing, preprocessing, transformations).
 - **Multithreading:** I/O-bound: loading json files from disk or network is slow.